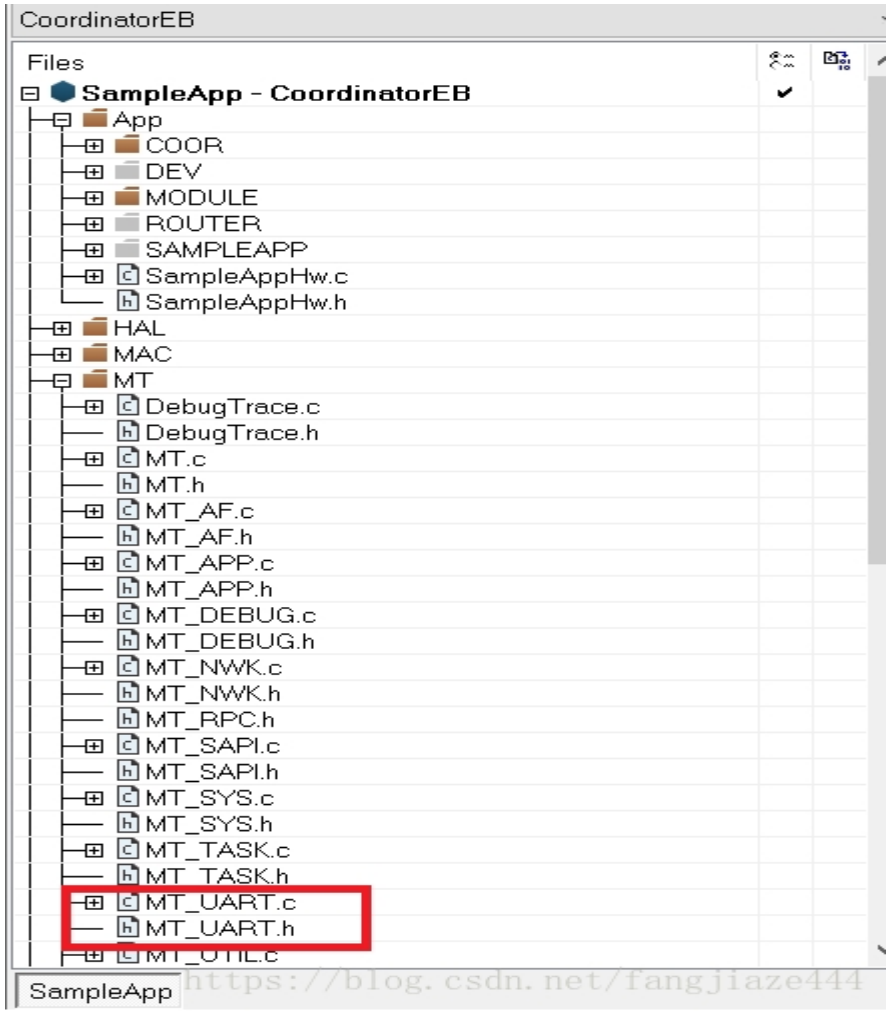
协议栈的串口透传

四川华迪信息技术有限公司

Version15.0

串口透传程序设计

MT层的串口API文件是MT_UART.c和MT_UART.h。如下图：



串口透传程序设计

一、串口初始化

1、在APP层SampleApp.c文件中添加

#include “MT_UART.H”

2、在SampleApp.c文件中的函数

void SampleApp_Init(uint8 task_id)添加语句

MT_UartInit();//串口初始化

MT_UartRegisterTaskID(task_id);//串口任务注册

这样我们在程序中就可以使用串口了。

MT_UartInit()函数在MT层的MT_UART.C中，主要修改波特率和流控制这两项

串口透传程序设计

波特率修改：

uartConfig.baudRate= MT_UART_DEFAULT_BAUDRATE;

流控制修改

uartConfig.flowControl = MT_UART_DEFAULT_OVERFLOW;

有关串口读写的函数：

读串口数据

uint16 HalUARTRead(uint8 port, uint8 *buf, uint16 len)

写串口数据

uint16 HalUARTWrite(uint8 port, uint8 *buf, uint16 len)

二、串口回传函数

串口透传程序设计

协议栈中CC2530接收计算机串口发送的数据是在串口回传函数中进行的。回传函数在MT_UART.C中的void MT_UartInit ()中 如图：

```
void MT_UartInit ()
{
  halUARTCfg_t uartConfig;

  /* Initialize APP ID */
  App_TaskID = 0;

  /* UART Configuration */
  uartConfig.configured           = TRUE;
  uartConfig.baudRate             = MT_UART_DEFAULT_BAUDRATE;
  uartConfig.flowControl          = MT_UART_DEFAULT_OVERFLOW;
  uartConfig.flowControlThreshold = MT_UART_DEFAULT_THRESHOLD;
  uartConfig.rx.maxBufSize        = MT_UART_DEFAULT_MAX_RX_BUFF;
  uartConfig.tx.maxBufSize        = MT_UART_DEFAULT_MAX_TX_BUFF;
  uartConfig.idleTimeout          = MT_UART_DEFAULT_IDLE_TIMEOUT;
  uartConfig.intEnable            = TRUE;
#if defined (ZTOOL_P1) || defined (ZTOOL_P2)
  uartConfig.callBackFunc         = MT_UartProcessZToolData;
```

串口回传函数

串口透传程序设计

MT_UartProcessZToolData功能：

这个函数处理的就是一个串口不断读取的函数，在注册了这个MT_UART后，这个函数会被不断的被调用，相当于事件一样，当串口有数据时，就处理这个数据并发送到上层即我们的应用层

TI的通讯协议串口数据具有以下格式：

0xFE, DataLength, CM0, CM1, Data payload, FCS

0xFE：数据帧头

DataLength： Datapayload 的数据长度，以字节计，低字节在前；

CM0：命令低字节；

CM1：命令高字节； (ZTOOL 软件就是通过发送一系列命令给 MT 实现和协议栈交互)

Data payload：数据帧具体的数据，这个长度是可变的，但是要和ataLength一致；

FCS ：校验和，从 DataLength 字节开始到 Data payload 最后一个字节所有字节的异或按字节操作；

串口透传程序设计

- TI协议处理过程：

1、 接收串口数据，判断起始码是否为 0xFE

2、 得到数据长度然后给数据包 pMsg 分配内存

3、 给数据包 pMsg 装数据

4、 打包成任务发给上层 OSAL 待处理

5、 释放数据包内存

这样处理太复杂。

串口透传程序设计

三、回传函数的改装

回传函数按照TI的协议来写，太过复杂，不适用。我们可以做一个回传函数

void MT_UartProcessZToolData (uint8 port, uint8 event)

把该函数换成我们自己的串口处理函数

```
void MT_UartProcessZToolData ( uint8 port, uint8 event )
{
   uint8 flag=0,i,j=0;    //flag是判断有没有收到数据，j记录数据长度
   uint8 buf[128];     //串口buffer最大缓冲默认是128，我们这里用128.
  (void)event;         // Intentionally unreferenced parameter
  while (Hal_UART_RxBufLen(port)) //检测串口数据是否接收完成
     {
        HalUARTRead (port,&buf[j], 1);  //把数据接收放到buf中
        j++;                            //记录字符数
        flag=1;                        //已经从串口接收到信息
     }
```

串口透传程序设计

```
    if(flag==1) //已经从串口接收到信息
    {
    /* Allocate memory for the data */
    //分配内存空间，为结构体内容+数据内容+1个记录长度的数据
pMsg = (mtOSALSerialData_t *)osal_msg_allocate( sizeof
( mtOSALSerialData_t )+j+1);
    pMsg->hdr.event = SPI_INCOMING_ZTOOL_PORT;//定义事件
    pMsg->msg = (uint8*)(pMsg+1); // 把数据定位到结构体数据部分
    pMsg->msg [0]= j;                //给上层的数据第一个是长度
    for(i=0;i<j;i++)                  //从第二个开始记录数据
    pMsg->msg [i+1]= buf[i];
    osal_msg_send( App_TaskID, (byte *)pMsg );  //登记任务，发往上层
    /* deallocate the msg */
    osal_msg_deallocate ( (uint8 *)pMsg );      //释放内存
    }
 }
```

串口透传程序设计

这样就能成功把消息数据通过osal_msg_send(App_TaskID,(byte*)pMsg)发送给应用层，也就是我们刚刚注册该MT_UART的这层，换句话说就是刚刚调用了MT_UartRegisterTaskID(task_id)的这层。理论上pMsg这个变量可以是任何类型的变量，包括任何的结构体变量，但是，这样的变量上在协议栈中有一个规范，就是必须数据头要有一个属性是osal_event_hdr_t的类型，记住这个osal_event_hdr_t类型，它是消息传递的关键。以上代码首先我们先看pMsg是一个mtOSALSerialData_t类型。其原型如下：

串口透传程序设计

```
typedef struct
{
  osal_event_hdr_t  hdr;
  uint8             *msg;
} mtOSALSerialData_t;
```

这个类型是MT_UART.h已经定义好的，并非自定义。当然也可以自己定义，当理解了大致的消息机制的框架我们自己定义类似的结构体。分析这个结构体，mtOSALSerialData_t里面有个osal_event_hdr_t hdr，这个上文提过是消息传递的关键，我们跟踪它，可以看到函数原型如下：

```
typedef struct
{
  uint8  event;
  uint8  status;
} osal_event_hdr_t;
```

串口透传程序设计

根据前面代码块的描述pMsg->hdr.event = SPI_INCOMING_ZTOOL_PORT,就是该事件的ID，status我们并没有给它赋值，因为此时并没用到，这里我如何分配内存的给pMsg的呢，该实现方式是：

```
pMsg = (mtOSALSerialData_t *)osal_msg_allocate( sizeof( mtOSALSerialData_t )+j+1)
```

mtOSALSerialData_t pMsg

Uint8 event=CMD_SERIAL_MSG
Uint8 state(未使用)
Uint8* msg
6
'1'
'2'
'3'
'4'
'5'
'6'

https://blog.csdn.net/fangjiaze444

此时我们把这样的数据通过osal_msg_send(App_TaskID,(byte*)pMsg)发送给应用层。

串口透传程序设计

接下来应用层如何接收呢，其实该事件发送应用层会触发应用层的SYS_EVENT_MSG系统事件。在我们的应用层里接收到该任何事件都会先把数据包先强制转换成afIncomingMSGPacket_t的类型，然后判断该判断这个数据头的event事件是什么，因此前面提到，其实发送任何数据都没问题，但是前提是数据头必须要有个osal_event_hdr_t类型的变量

串口透传程序设计

```
typedef struct
{
  osal_event_hdr_t hdr;      /* OSAL Message header */
  uint16 groupId;            /* Message's group ID - 0 if not set */
  uint16 clusterId;          /* Message's cluster ID */
  afAddrType_t srcAddr;      /* Source Address, if endpoint is STUBAPS_INTER_PAN_EP,
                                it's an InterPAN message */
  uint16 macDestAddr;        /* MAC header destination short address */
  uint8 endPoint;            /* destination endpoint */
  uint8 wasBroadcast;        /* TRUE if network destination was a broadcast address */
  uint8 LinkQuality;         /* The link quality of the received data frame */
  uint8 correlation;         /* The raw correlation value of the received data frame */
  int8  rssi;                /* The received RF power in units dBm */
  uint8 SecurityUse;         /* deprecated */
  uint32 timestamp;          /* receipt timestamp from MAC */
  uint8 nwkSeqNum;           /* network header frame sequence number */
  afMSGCommandFormat_t cmd; /* Application Data */
} afIncomingMSGPacket_t;
```

串口透传程序设计

虽然后面的数据会跟我们发送的mtOSALSerialData_t类型不一样，只要我们不去随便触碰不应该存在的数据就行了。因为我们知道后面数据是不存在groupid，clusterld，srcAddr等等，我们只知道这个数据包的存储空间结构是我们发送的msg和6和“123456”.因此，我们在测试这个数据时要再次转换回来

串口透传程序设计

```
uint16 SampleApp_ProcessEvent( uint8 task_id, uint16 events )
{
  afIncomingMSGPacket_t *MSGpkt;
  mtOSALSerialData_t * UartMsg;
  (void)task_id;  // Intentionally unreferenced parameter
  if ( events & SYS_EVENT_MSG )
  {
    MSGpkt = (afIncomingMSGPacket_t *)osal_msg_receive( SampleApp_TaskID );
    while ( MSGpkt )
    {
      switch ( MSGpkt->hdr.event )
      {
        // Received when a key is pressed
        case KEY_CHANGE:
          SampleApp_HandleKeys( ((keyChange_t *)MSGpkt)->state,
((keyChange_t *)MSGpkt)->keys );
          break;
```

串口透传程序设计

然后再switch里面添加case SPI_INCOMING_ZTOOL_PORT：，测试代码块我们可以以下实现：

```
case SPI_INCOMING_ZTOOL_PORT:
        UartMsg = (mtOSALSerialData_t *)MSGpkt;
        HalUARTWrite(0,&UartMsg->msg[1],UartMsg->msg[0]);
        break;
```

UartMsg在函数的前面定义如下:

```
mtOSALSerialData_t * UartMsg;
```

也可直接写成函数形式

```
case SPI_INCOMING_ZTOOL_PORT:
          SerialCMD((mtOSALSerialData_t *)MSGpkt);
          break;
```

串口透传程序设计

- 大家是不是很奇怪怎么在协议栈里找不到这个函数，当然了，我们那边只把他打包了，然后登记任务，这个包是我们自己的，想怎么处理当然由自己决定。

串口透传程序设计

串口透传相关的函数：

串口初始化函数

```
void MT_UartInit ()
```

串口注册函数

```
void MT_UartRegisterTaskID( byte taskID )
```

写串口函数：

```
uint16 HalUARTWrite(uint8 port, uint8 *buf, uint16 len)
```

读串口函数：

```
uint16 HalUARTRead(uint8 port, uint8 *buf, uint16 len)
```

串口透传程序设计

检测串口是否有数据函数

uint16 Hal_UART_RxBufLen(uint8 port)

信息管理API

（1)osal_msg_allocate ( )

功能描述：

为信息分配缓存空间、任务调用或函数被调用的时候，该空间被信息填充或调用发送信息函数osal_msg_send()发送缓存空间的信息到其他任务。

原型：

byte *osal_msg_allocate(uint16 len)

参数：

len ：消息的长度释放内存：

uint8 osal_msg_deallocate(uint8 *msg_ptr)

一个任务发送一个命令或数据信息到其他任务或处理元素中。

uint8 osal_msg_send(uint8 destination_task, uint8 *msg_ptr)

串口透传程序设计

（2）osal_msg_deallocate( )

功能描述：用于收回缓冲区

原型：　　byte osal_msg_deallocate(byte *msg_ptr)

参数：　　Msg_ptr ：指向将要收回的缓冲区的指针

（3） osal_msg_send( )

功能描述：

任务调用这个函数以实现发送指令或数据给另一个任务或处理单元。目标任务的标识必须是一个有效的系统任务，当调用osal_create_task ( )启动一个任务时，将会分配任务标识。

osal_msg_send()也将在目标任务的事件列表中设置SYS_EVENT_MSG

原型：　　byte osal_msg_send(byte destination_task, byte *msg_ptr)

参数：　　destination_task ：目标任务的标识

msg_ptr ： 指向消息缓冲区的指针，必须是osal_msg_allocate ( )函数分配的有效的数据缓存

Q&A

THANK YOU

- 请提问！